Текст программы

```
from operator import itemgetter


class StoredProcedure:
    """Хранимая процедура"""

    def __init__(self, id, name, time_ms, db_id):
        self.id = id
        self.name = name
        self.time_ms = time_ms
        self.db_id = db_id


class Database:
    """База данных"""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class ProcedureDatabase:
    """Связь хранимых процедур и баз данных многие-ко-многим"""

    def __init__(self, db_id, procedure_id):
        self.db_id = db_id
        self.procedure_id = procedure_id


# Базы данных
databases = [
    Database(1, 'база данных продаж'),
    Database(2, 'база данных склада'),
    Database(3, 'архив заказов'),
    Database(4, 'аналитика'),
]

# Хранимые процедуры
procedures = [
    StoredProcedure(1, 'Обновить остатки', 120, 2),
    StoredProcedure(2, 'Сформировать отчет', 80, 1),
    StoredProcedure(3, 'Закрыть заказ', 50, 1),
    StoredProcedure(4, 'Очистить архив', 30, 3),
]

procedures_databases = [
    ProcedureDatabase(1, 2),
    ProcedureDatabase(1, 3),
    ProcedureDatabase(2, 1),
    ProcedureDatabase(2, 2),
    ProcedureDatabase(4, 2),
]


def main():
    """Основная функция"""

    # Соединение данных один-ко-многим
    one_to_many = [(p.name, p.time_ms, d.name)
        for d in databases
        for p in procedures
        if p.db_id == d.id]
```

```
    # Соединение данных многие-ко-многим
    many_to_many_temp = [(d.name, pd.db_id, pd.procedure_id)
        for d in databases
        for pd in procedures_databases
        if d.id == pd.db_id]

    many_to_many = [(p.name, p.time_ms, db_name)
        for db_name, db_id, procedure_id in many_to_many_temp
        for p in procedures if p.id == procedure_id]

    print('Задание A1')
    res_11 = sorted(one_to_many, key=itemgetter(2))
    print(res_11)

    print('\nЗадание A2')
    res_12_unsorted = []
    # Перебираем все базы данных
    for d in databases:
        # Список процедур базы данных
        d_procedures = list(filter(lambda i: i[2] == d.name, one_to_many))
        # Если база данных не пустая
        if len(d_procedures) > 0:
            # Время выполнения процедур в миллисекундах
            d_times = [time_ms for _, time_ms, _ in d_procedures]
            # Суммарное время выполнения
            d_times_sum = sum(d_times)
            res_12_unsorted.append((d.name, d_times_sum))

    # Сортировка по суммарному времени выполнения
    res_12 = sorted(res_12_unsorted, key=itemgetter(1), reverse=True)
    print(res_12)

    print('\nЗадание A3')
    res_13 = {}
    # Перебираем все базы данных
    for d in databases:
        if 'база данных' in d.name:
            # Список процедур базы данных
            d_procedures = list(filter(lambda i: i[2] == d.name, many_to_many))
            # Только названия процедур
            d_procedures_names = [name for name, _, _ in d_procedures]
            # Ключ - база данных, значение - список процедур
            res_13[d.name] = d_procedures_names

    print(res_13)


if __name__ == '__main__':
    main()
```

Результаты выполнения

```
Задание A1
[('Очистить архив', 30, 'архив заказов'), ('Сформировать отчет', 80, 'база данных продаж'),
('Закрыть заказ', 50, 'база данных продаж'), ('Обновить остатки', 120, 'база данных склада')]

Задание A2
[('база данных продаж', 130), ('база данных склада', 120), ('архив заказов', 30)]

Задание A3
{'база данных продаж': ['Сформировать отчет', 'Закрыть заказ'], 'база данных склада':
['Обновить остатки', 'Сформировать отчет']}
```